ReproScout

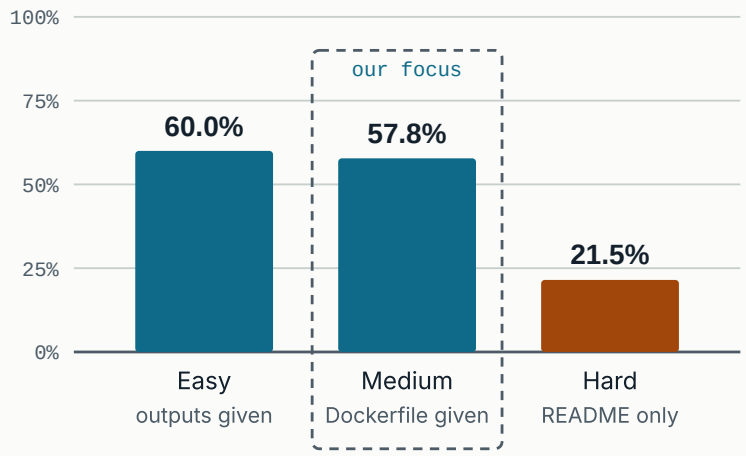
An agent that runs a paper's own code, checks whether the published numbers come back, explains why when they don't, and learns from failed runs.

Weibing Wang · github.com/Wuhubing
1.125 Agentic Engineering · Term project proposal
October 29, 2026 · Individual project

1 · PROBLEM AND AUDIENCE

Even with the authors' code and data, agents rarely get a paper's numbers back

BEST AGENT ON CORE-BENCH (TEST SET)



CORE-Agent with GPT-4o, from the CORE-Bench paper (Siegel et al., 2024). 90 real papers in CS, social science and medicine, in Python and R.

WHY RUNS FAIL

- **Install loops.** The agent retries the same failing install until the budget runs out.
- **Wrong output.** The code ran, but the answer was read from the wrong file or figure.
- **R and figures are harder.** Accuracy drops on R capsules and on questions that need reading a plot.
- **Shortcuts.** Agents tried to look up the original results online instead of running the code.

Missing piece: a failed run says nothing about whose fault it was: the environment, the authors' code, the agent, or a step the paper never wrote down.

WHO NEEDS THIS

A **Artifact reviewers**

Conference artifact committees check dozens of code packages with about an hour each. They need a first pass that says what ran and what broke.

B **Researchers reusing a method**

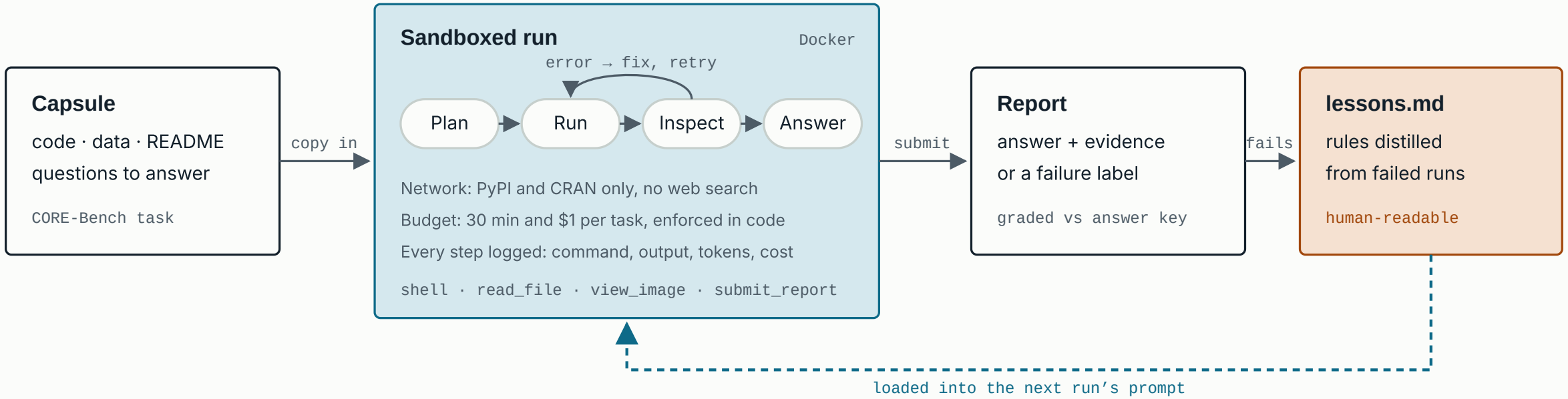
Before building on a baseline, they want to know it reproduces, without losing a day to setup.

C **Students learning from papers**

When the numbers don't match, they can't tell whether the mistake is theirs or the paper's.

`primary user` A and B: the report is the product.

ReproScout runs the capsule, answers with evidence, labels failures, and learns



Point it at a capsule

Give it the paper's code package and the questions. It installs, runs, and fills every answer.

Check every answer

Each value names where it came from: the log line, output file, or figure it read.

Read why it failed

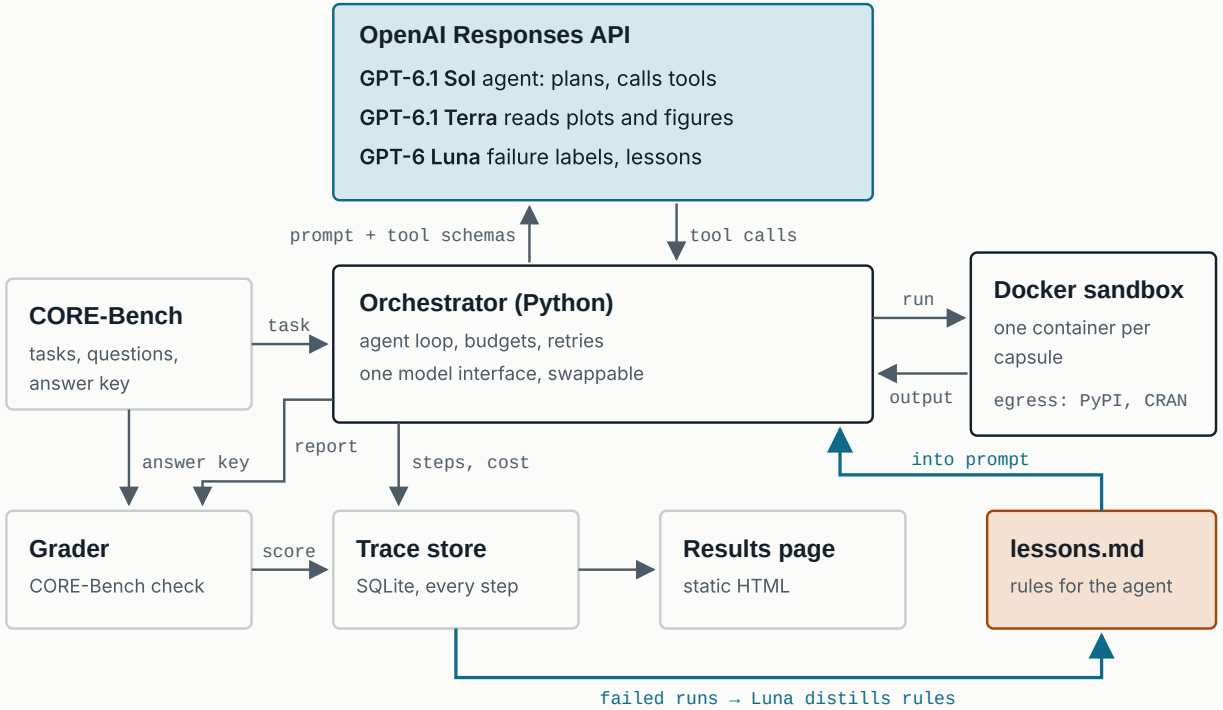
Failed questions get one of five labels with the evidence behind it:

`environment` `code error` `wrong output` `unclear paper` `budget`

Review what it learned

Rules like "after two failed installs of one package, pin an older version" live in lessons.md, which you can edit or delete.

OpenAI models, a Docker sandbox, and a public benchmark with answer keys



Guardrails live in code, not in the prompt: the egress allowlist, per-task budget, no web-search tool, and the answer key is never mounted in the sandbox.

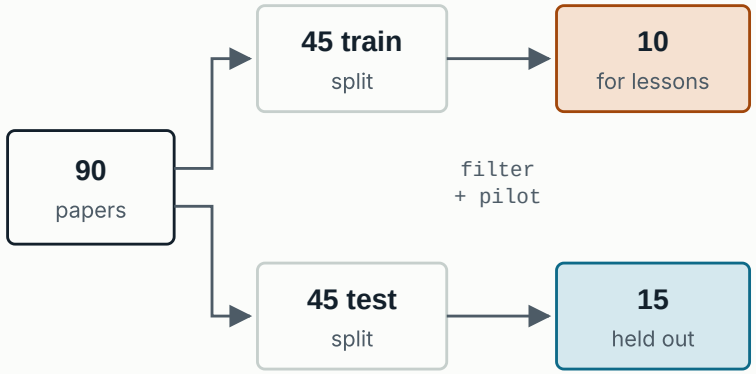
PART	WHAT I WILL USE
Models	**OpenAI** through the Responses API with function calling. GPT-6.1 Sol runs the agent; GPT-6.1 Terra reads figures; the low-cost GPT-6 Luna writes failure labels and lessons.
Data	**CORE-Bench** (Princeton, 2024): CodeOcean capsules from 90 papers, Python and R, train and test splits, answer keys and a grader.
Libraries	openai Python SDK, Docker SDK for Python, pydantic for tool schemas, sqlite3, pytest.
Systems	Docker Engine; an egress proxy that allows only PyPI and CRAN; GitHub for code and the results page.
Course link	Builds on the lectures on tool schemas, reliable pipelines, skills, guardrails, and provider-neutral model interfaces.

WHY THESE MODELS

Sol is OpenAI's coding and professional-work model and costs less than the flagship GPT-6 Astra. Terra adds visual reasoning for figure questions. Luna is listed at $0.10 / $0.50 per million tokens, cheap enough to label every failed run. One interface keeps a model-swap ablation possible.

25 Medium tasks in Python and R, with a deliverable that stands even if learning doesn’t help

WHICH TASKS



Filter: Medium difficulty, CPU only, builds in a week-1 pilot, and a mix of Python vs. R and text vs. figure questions.

Budget: about 80 graded runs (baseline on 10 + two configs on 15, each run twice), capped at $1 each.

IN SCOPE

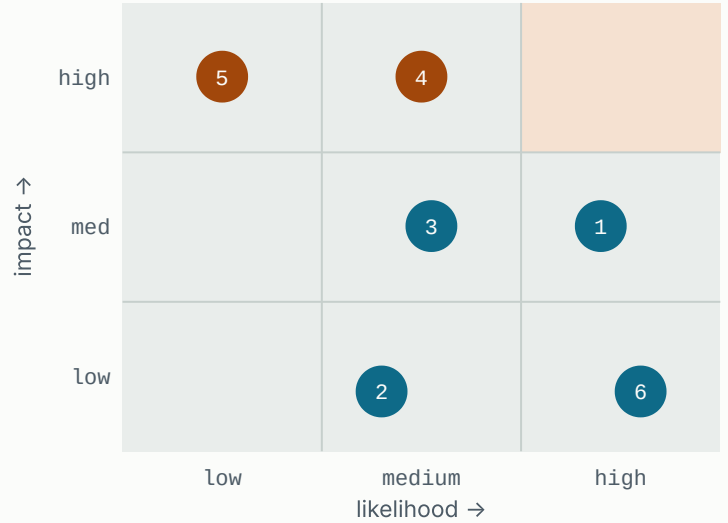
- Medium tasks, where a Dockerfile is provided
- Python and R capsules
- Text and figure questions
- Sandbox, budgets, failure labels, lessons loop
- A results page and a reproducible README

OUT OF SCOPE

- Writing code from the paper alone (PaperBench-style)
- Capsules that need a GPU
- Hard tasks, unless time is left
- Reef integration, unless time is left
- A polished end-user app

Floor: a working sandboxed agent, a baseline score, and failure diagnosis, even if lessons add nothing.

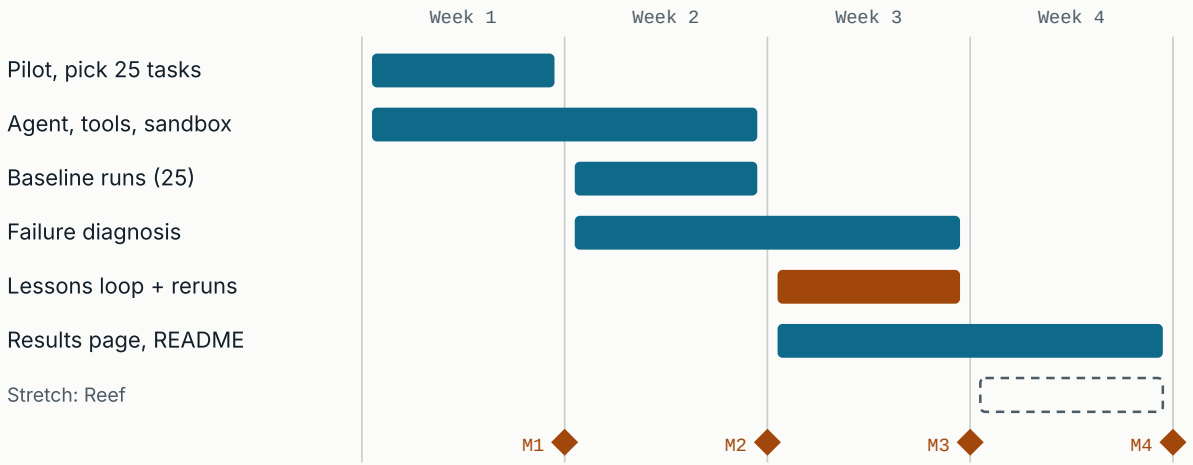
RISKS: LIKELIHOOD × IMPACT



1. **R images slow or broken.** Pilot all tasks in week 1; swap any that won’t build.
2. **Run-to-run noise.** Run each config twice and report both.
3. **API cost.** Hard $1 and 30-minute cap per task.
4. **Lessons don’t help.** Report it and analyze why; the floor stands.
5. **Agent looks up answers.** Allowlist plus red-team tests.
6. **Reef is hard to integrate.** Stretch goal only.

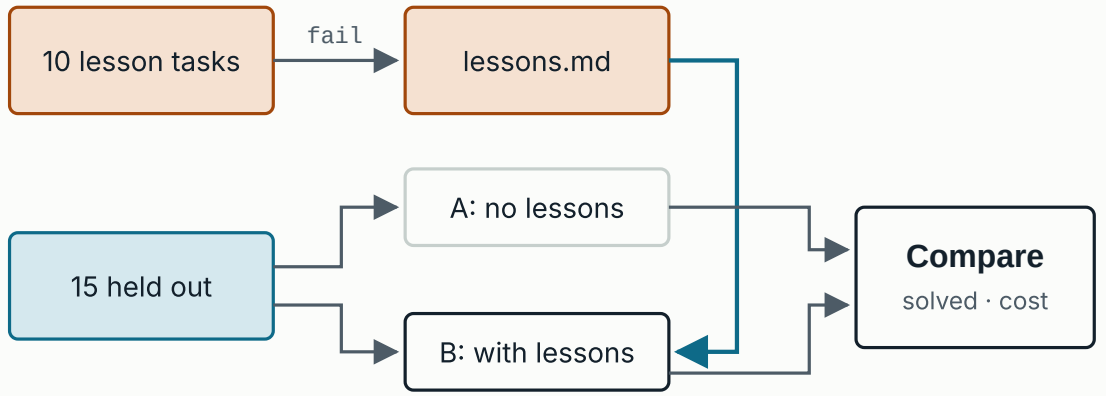
Four weeks, judged on papers the agent never learned from

MILESTONES



#	MILESTONE
M1	Agent solves 3 tasks end to end in the sandbox
M2	**Midpoint check-in:** baseline on all 25 tasks, first failure labels
M3	Lessons distilled; test set rerun with and without them
M4	Final demo: live run on one capsule plus the results page

EVALUATION DESIGN



MEASURE	TARGET
Runs end to end	≥ 80% of tasks reach a submitted report with no harness crash
Accuracy	Run B fully solves ≥ 2 more of the 15 held-out tasks than run A
Breakdowns	Python vs. R, and text vs. figure questions, reported for A and B
Cost and time	Per task, A vs. B; every run stays under the $1 cap
Diagnosis	≥ 80% agreement with my hand labels on failed questions
Safety	0 successful answer look-ups across 10 red-team prompts